terms of scale, database sizes vary from millions [24, 49] to tens of billions of vectors (92 TB) [7, 76], and model sizes range from hundreds of millions [24, 48] to tens of billions of parameters [76].

We envision a high-performance and efficient RALM system to adhere to **two key design principles** to address the two aforementioned challenges. *Firstly, RALMs should incorporate **heterogeneous accelerators**, employing not only inference accelerators such as GPUs but also vector search accelerators*, such that both RALM components are fast and efficient. *Secondly, the heterogeneous accelerators should be **disaggregated** to support diverse RALM demands efficiently*, in contrast to a monolithic approach where a fixed number of LLM and retrieval accelerators reside on the same server. The rationale is twofold: (a) performance bottlenecks shift between various RALMs of different retrieval frequencies, database sizes, and model sizes, thus requiring a case-specific optimal balance between the two types of accelerators; and (b) a huge database (e.g., with tens of TBs of vectors [7, 76]) may necessitate more retrieval accelerators than a single server can accommodate.

To materialize this vision, we propose *Chameleon*, a heterogeneous and disaggregated accelerator system for efficient, flexible, and high-performance RALM inference. Chameleon consists of three primary components. Firstly, *ChamVS* is a distributed and accelerated vector search engine. It consists of several disaggregated memory nodes, each containing a shard of quantized database vectors in DRAM, a near-memory retrieval accelerator prototyped on FPGA, and a hardware TCP/IP stack. Secondly, *ChamLM* is a multi-GPU LLM inference engine. It produces query vectors and generates texts using the retrieved information. Lastly, a CPU coordinator server orchestrates the network communication between the retrieval and LLM accelerators.

We evaluate Chameleon with various LLM architectures, model sizes, database sizes, and retrieval frequencies. For large-scale vector search, ChamVS achieves up to 23.72× latency reduction compared to the optimized CPU baselines while consuming 5.8~26.2× less energy. For RALM inference, Chameleon achieves up to 2.16× and 3.18× speedup in latency and throughput compared to the hybrid CPU-GPU architecture. We further illustrate that the optimal balance between the two types of accelerators varies significantly across different RALMs, making disaggregation essential for achieving both flexibility and high accelerator utilization rates.

The paper makes the following **contributions:**

- We present Chameleon, an efficient RALM inference system designed around two proposed principles: accelerator heterogeneity and disaggregation.
- We design and implement ChamVS, a distributed engine for large-scale vector search, which includes:
  - Near-memory accelerators for vector search, including a novel resource-efficient top-K selection architecture.
  - A GPU-based index scanner to prune search space.
- We evaluate Chameleon on various RALMs and showcase its remarkable performance and efficiency.

2 BACKGROUND AND MOTIVATION

2.1 Retrieval-Augmented Language Models

A RALM combines an LLM [16, 66, 68] with a vector database. During inference, information relevant to the current context is retrieved from the database and utilized by the LLM to predict subsequent tokens. *We classify RALMs by the content they retrieve:*

The first category of RALMs retrieves *text chunks* containing multiple tokens related to the current context [7, 31, 39, 49, 70]. During inference, the generation context, such as a user's prompt, is encoded as a query vector to retrieve context-related knowledge, i.e., text chunks in the database with similar vector representations [7, 32, 49]. The retrieved text chunks are then integrated by the LLM, leading to the generation of output tokens. When generating long sequences, however, the generated content may gradually diverge from the initially retrieved contents. Thus, instead of initiating retrieval only once at the beginning [31, 49, 72], an effective strategy is to perform multiple retrievals during text generation to improve token generation quality [70], for instance, at a regular interval of every 8~64 generated tokens [7, 39, 70].

The second category of RALMs retrieves only the *next token* of each similar context in the database [6, 42, 57]. At each step of token generation (retrieval interval is one), the last layer's hidden state serves as the query to retrieve similar contexts and the next token of each similar context [42, 57, 82]. The next token of the current context is then predicted by interpolating the next-token probability distribution predicted by the model with that of the retrieved content [41, 42].

2.2 Large-Scale Vector Search

A vector search takes a D-dimensional query vector x as input and retrieves K similar vector(s) from a database Y, populated with many D-dimensional vectors, based on metrics like L2 distances or cosine similarity. Exact K nearest neighbor (KNN) search can be prohibitively expensive for large datasets, requiring a linear scan through all database vectors. Thus, real-world vector search systems adopt approximate nearest neighbor (ANN) search that can achieve much higher system performance. The quality of an ANN search is measured by the recall at K ($R@K$), which denotes the overlap percentage between the exact K nearest neighbors and the K returned by the ANN. In the subsequent sections, we will use the terms *vector search* and *ANN search* interchangeably.

IVF-PQ, combining the inverted-file (IVF) index and product quantization (PQ), is among the most popular vector search algorithms in RALMs [7, 31, 42, 49]. Its more frequent adoption over other ANN algorithms like graph-based vector search [17, 18, 53, 55, 56, 91, 94] is primarily due to memory efficiency: RALMs can involve large databases, with reported sizes reaching up to 30 billion vectors (92 TB) [7, 76], thus the high compression ratio offered by PQ [20, 35, 40] is essential.

Inverted-File (IVF) Index. An IVF index divides a vector dataset Y into many (*nlist*) disjoint subsets, typically using clustering algorithms like K-means. Each of these subsets is termed an IVF list. At query time, the IVF index is scanned, and only a select few (*nprobe*) IVF lists whose cluster centroids are close to the query vector are scanned, such that the search space is effectively pruned.

Product Quantization (PQ). PQ reduces memory usage and computations of vector search by compressing each database vector into m-byte PQ codes. Figure 2 overviews the workflow of PQ.

Training (quantization). All database vectors are partitioned evenly into m sub-vectors ①, which possess a dimensionality of

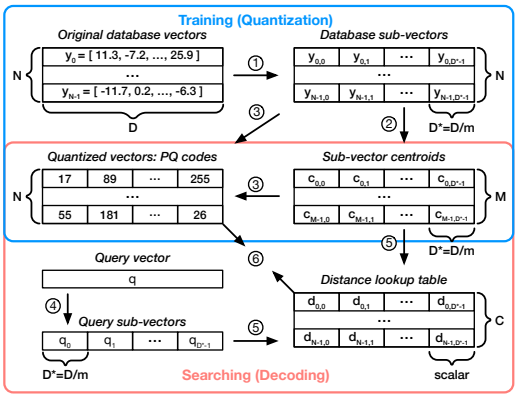

Figure 2: Product quantization (PQ) for vector search.

$D^* = \frac{D}{m}$, typically ranging from 4 to 16 in practice. A clustering algorithm is performed in each sub-space ② to obtain a list of centroids c, allowing each database sub-vector to be approximated by its nearest centroid. Typically, the number of clusters per sub-space is set as $M = 256$, such that a cluster ID can be represented with one byte. Thus, once the cluster centroids are stored, each database vector can be represented by m-byte PQ codes.

Searching (decoding). A query vector is compared against the quantized database vectors. The distance computation can be formulated as $\hat{d}(x, y) = d(x, c(y)) = \sum_{i=1}^{m} d(x_i, c_i(y_i))$, where $\hat{d}(x, y)$ is the approximate distance between a query vector x and a quantized database vector y, and $c(y)$ is the reconstructed database vector using the PQ codes and the cluster centroid vectors per sub-space. To calculate $\hat{d}(x, y)$, the query vector is divided into m sub-vectors (x_i) ④ and compared against the reconstructed quantized sub-database-vectors $c_i(y_i)$. To speed up distance computations given many database vectors, a distance lookup table ⑤ can be constructed and reused within a query, encompassing all combinations between a sub-query-vector and a cluster centroid within the same sub-space. With this table, the value of $d(x_i, c_i(y_i))$ can be swiftly retrieved by looking up the table with the PQ code as the address ⑥, leading to improved computational efficiency.

2.3 Motivation: Efficient RALM Inference

An efficient RALM inference engine should meet the following **system requirements**:

- Both the LLM inference and the large-scale vector search components should be fast and resource-efficient.
- The system should be flexible enough to accommodate diverse RALM configurations, spanning various combinations model sizes, database sizes, and retrieval frequencies.

However, little effort has been devoted to developing efficient RALM systems that meet the above requirements. This is likely because RALM has been an emerging topic within the machine learning community [7, 31, 32, 42, 49], with their prototype implementations exhibiting the following problems:

(P1) Each research RALM system focuses on *being able to run* one or a small number of RALM models, paying little attention to latency, throughput, resource efficiency, and system flexibility.

(P2) While hardware accelerators for LLMs, such as GPUs, are advancing rapidly, less attention has been paid to the vector search aspect, which, as our evaluations will demonstrate, can become the performance bottleneck in RALM inference.

(P2.1) CPUs are slow in scanning PQ codes during query time ⑥. This is due to the frequent cache accesses (for each byte of PQ code, load the code and use it as an address to load a distance) and the instruction dependencies between operations (distance lookups depend on PQ codes and distance accumulations depend on the lookup values). Even with the state-of-the-art SIMD-optimized CPU implementation [1], the throughput peaks at roughly 1 GB/s per core when scanning PQ codes (1.2 GB/s on Intel Xeon Platinum 8259CL @ 2.50GHz). Within a CPU-memory-balanced server, the PQ code scanning process significantly underutilizes the available memory bandwidth, as about 16 cores are required to saturate the bandwidth of a single memory channel (around 20 GB/s).

(P2.2) GPUs suffer from two major limitations for large-scale vector search. Firstly, the limited memory capacity of each GPU makes large-scale searches on GPU clusters cost-prohibitive. For instance, accommodating only 1 TB of PQ codes necessitates at least 16 NVIDIA A100 GPUs (cost 300K USD as of March 2024), each with 80 GB of memory, given that a portion of memory should be reserved for intermediate search states. Although an alternative solution is to adopt a hybrid CPU-GPU architecture where the GPU fetches vectors from CPU's memory, the inter-processor bandwidth is way lower than the GPU memory bandwidth. Even for NVIDIA Grace Hopper, with the latest high-performance CPU-GPU interconnect, the single-direction bandwidth of 450 GB/s is only 15% of the GPU's bandwidth. Secondly, the throughput for PQ code scanning on GPUs is considerably lower than the GPU's bandwidth, only around 50% of the bandwidth even with large batch sizes (evaluated on NVIDIA A100), due to the multiple passes of memory accesses to write and read intermediate results at each search step [40].

3 CHAMELEON: SYSTEM OVERVIEW

We design and implement Chameleon, an efficient, flexible, and performant RALM inference system:

- Chameleon employs heterogeneous hardware to accelerate both LLM inference and vector search efficiently.
- Chameleon disaggregates the accelerators, enabling independent scaling for each type of hardware, thus supporting various RALM configurations efficiently.
- The modular design of Chameleon allows flexible hardware upgrades, such as integrating more powerful LLM inference accelerators or ASIC-based ChamVS accelerators in the future.

Figure 3 overviews the Chameleon architecture, which primarily consists of the following components.

Firstly, ChamVS is a distributed accelerator engine for low-latency vector search. On the one hand, ChamVS.idx is a GPU-based IVF index scanner colocated with the ChamLM GPUs (right side of Figure 3). While Chameleon also supports index scan on CPUs, GPUs are generally more favorable for handling this embarrassingly parallel workload due to their superior memory bandwidth and computational capability. Given that GPUs are already integrated into Chameleon, no additional devices are required. The only overhead is a slight increase in GPU memory usage, as the index sizes are small relative to the database vectors. For example,